Unit 2

Python Programming Crash Course

What is Unit 2 About?

In this unit, we will dive deeper into Python programming! Now that you have set up your workspace, it's time to build a strong foundation in coding.

Whether you are new to programming or have some experience, this unit will help you develop essential coding skills needed for data science. We will build off the ideas presented in Unit 1.

You will begin applying Python to real-world data and problem-solving tasks.

Learning Outcomes

01
Understand core concepts: variables, loops, conditionals, and functions.

02
Write and structure functional Python programs.

03
Process and analyze data using Python.

What Will We Do in Unit 2?

1. Further Solidify Python Fundamentals

You'll build on your foundation in Python by mastering:

Variables & Data Types
Store and manipulate numbers, text, and collections of data.

Operators & Expressions
Perform calculations and logical operations.

Loops & Conditionals
Automate tasks and make decisions in code.

Functions
Write reusable blocks of code to simplify programs.

Lists & Dictionaries
Organize and structure data efficiently.

What Will We Do in Unit 2?

2. Write Programs & 3. Mini-Project

2. Write Your First Programs

Through hands-on activities and guided coding exercises, you'll practice:

- Writing Python scripts in Jupyter Notebooks.
- Debugging and fixing common coding errors.
- Breaking down problems and thinking like a programmer.

3. Complete a Mini-Project

Apply what you've learned to a small coding project. Examples include:

- A simple chatbot that responds to user input.
- A number guessing game using loops and conditionals.
- A basic data processing program using lists and dictionaries.

Python Programming Crash Course

Unit Timeline

Day 1–2	**Python basics:** Variables, data types, and expressions.
Day 3–4	**Logic & Flow:** Conditional statements (if, elif, else) and loops (for, while).
Day 5	**Functions:** Functions and modular programming.
Day 6	**Organizing Data:** Lists and dictionaries for organizing data.
Day 7	**Mini-project:** Apply your Python skills to solve a problem!

Unit 2: Python Programming Crash Course

Skills & Significance

What Skills Will You Learn?

By the end of this unit, you will be able to:

- Automate tasks and solve problems with Python.
- Use loops and conditionals for decision-making.
- Write organized, reusable code with functions.
- Manage structured data with lists and dictionaries.

Why Is This Important?

Python is the backbone of data science and AI. Mastering it allows you to analyze data and create algorithms—essential skills for business, engineering, and research careers.

What's Next?

Get ready for real-world datasets! In Unit 3, we'll dive into advanced data manipulation and analysis tools.

Objectives for Today

Review & Expand
We will review basic python concepts and expand on them

Experimental Coding
We will do some experimental coding

First Functions
We will make our first functions

Instruction - Python Programming (Review)

Writing and Running Code

What is Python Code?

Python is a language that allows us to write instructions for computers. Each line performs specific tasks like displaying text, calculating, or analyzing data.

Where to Write Code?

We use Jupyter Notebooks to:

- Write/run code in small cells.
- See results immediately.
- Mix code with text documentation.

How to Run Code?

Execution Steps:

1. **Click inside a code cell.**
2. **Type a command (e.g., `print("Hello")`).**
3. **Click the Run button OR**
4. **Press Shift + Enter.**

Python Hands-on Exercise

Try It Yourself! (Review)

Step 1: Write and Run Code

Copy and run the following code in a Jupyter Notebook cell. You should see the text "Welcome to Data Science!" appear immediately below.

PYTHON

```
print("Welcome to Data Science!")
```

Pro Tip:

Use the **Shift + Enter** shortcut to run your cell quickly!

Remember! It is important to actually type the material out!

Building muscle memory helps you understand and remember the material better.

Instruction - Jupyter Notebook Basics

Code vs. Markdown Cells (Review)

Jupyter Notebooks have two main types of cells:

Code Cells

- Contain Python code for execution.
- Running a cell shows processed output.

EXAMPLE

```
In [1]: 2 + 3
Out[1]: 5
```

Markdown Cells

- Contain text, headings, and images.
- Used to document and explain code.

EXAMPLE

My First Program

This program prints a welcome message.

Common Mistakes and How to Fix Them

Debugging Your Code (Review)

Mistake 1: Forgetting to Run the Code

If you write code but don’t see an output, make sure you have actually run the cell.

Mistake 2: Syntax Errors

If you see a red error message, check your code for typos or missing punctuation.

THE PROBLEM

This will cause an error because the closing quotation mark is missing.

```
# Error
print("Hello world!
```

```
# Correct
print("Hello world!")
```

Summary (Review)

Working with Cells

Python code is written and executed in Jupyter Notebook cells. Use code cells for Python and markdown cells for text.

Executing Code

Run your code using the Run button or the Shift + Enter keyboard shortcut.

Troubleshooting

Debug errors by carefully checking your syntax for typos or missing punctuation.

Activity - Review

Practice Writing & Running Code

Objective

In this activity, you will practice writing and running Python code in a Jupyter Notebook. You will:

- Start with simple Python commands
- Learn how to use different types of cells
- Troubleshoot and fix basic code errors

Activity - Step-by-Step Guide

Getting Started with Python

Step 1: Setup

Open Jupyter Notebook

Navigate to your notebook in GitHub Codespaces.

New Notebook:

- File > New Notebook
- Name it "Practice_Python.ipynb"

Step 2: Run Code

Write & Execute

Type this in a code cell:

```
print("Welcome to Python!")
```

Press **Shift + Enter** to run.

Expected Output:

```
Welcome to Python!
```

Step 3: Operations

Math in Python

Try these in separate cells:

```
8 + 2
```

```
15 / 3
```

```
5 * 4 - 1
```

What do you notice?

Python performs the calculations and displays the results.

Activity - Step-by-Step Guide

Working with Variables & Markdown

Step 4: Working with Variables

Variables store information in Python.

```
name = "Jordan"
age = 17
print("Hello, my name is", name, "and I am",
age, "years old.")
```

What happens?

- The name and age variables store text and numbers.
- The print() function combines them into a sentence.

Step 5: Markdown Cells

Jupyter Notebooks allow you to write explanations using Markdown Cells.

- Insert a new cell (+ button).
- Change type to **Markdown** (top dropdown).

Type this:

```
## My First Python Program
This notebook contains basic Python operations
and variables.
```

Press **Shift + Enter** to format it as text.

Activity - Step-by-Step Guide

Debugging & Coding Challenge

Step 6: Debugging Errors

Try running this incorrect code:

```
print("Python is great!)
```

Result: **SyntaxError**

Fix it by adding the closing quote:

```
print("Python is great!")
```

Challenge Task

Write Python code that:

- Asks for the user's favorite color.
- Stores it in a variable.
- Prints a message including their color.

Tip: Use the `input()` function to get user data.

Activity - Completion

Reflection & Submission

Reflection Questions

- What happens if you forget to run a code cell?
- How do you fix a SyntaxError?
- How can Markdown cells be useful in a Jupyter Notebook?

Save and submit your completed notebook.

Activity - Coding

Case Conversion Research

Assignment Task

Create commands that:

- Take in a statement as input.
- Print the statement in lowercase.

Success Criteria:

- Jupyter Notebook runs top-to-bottom.
- Use the "run-all" button.
- Test with multiple inputs.

Research & Guidance

A big part of programming is finding solutions through research. Research and testing are vital skills for Data Scientists.

Key Resource:

`.lower() method`

https://www.datacamp.com/tutorial/case-conversion-python

Note: Avoid using AI for this task; focus on building your independent research skills.

Instruction - Python Basics

Methods

What is a Method?

A method in Python is a function that belongs to an object and is used to perform specific actions on that object. Methods are like built-in tools that objects like strings, lists, or numbers use to do something useful.

Method vs. Function

Functions
Standalone blocks of code.

```
print("Hello!")
```

Methods
Belong to an object; used with dot (.) notation.

```
text.upper()
```

Example Code

```
text = "hello"

# Uses the .upper() method
print(text.upper())
```

Note: .lower() was used in the previous activity to change strings to lowercase.

Instruction - Python Basics

Using Methods in Python

Let’s look at some examples of methods for different types of objects.

String Methods

Modify text strings:

```
word = "python"
# word.upper() -> "PYTHON"
# word.lower() -> "python"
# word.capitalize() -> "Python"
```

List Methods

Add, remove, and sort elements in lists:

```
nums = [3, 1, 4, 1, 5]
# nums.append(9)
# nums.sort()
print(nums)
# [1, 1, 3, 4, 5, 9]
```

Input Methods

Modify user input:

```
name = input().strip()
# strip() removes spaces
print("Hello, " + name)
```

Instruction - Python Basics

Key Takeaways

Methods are powerful built-in tools in Python that allow you to perform actions on specific objects. They follow a consistent syntax and vary across different data types.

Core Definition

Methods are essentially functions that **belong to an object**. They are tied to the data they operate on.

Syntax Pattern

Access methods using **dot notation**:

```
object.method()
```

Always include parentheses, even if no arguments are passed.

Data Specificity

Different data types like **strings, lists, and numbers** have their own unique methods for common tasks.

Beyond built-in options, advanced Python involves creating custom methods through Object Oriented Programming (OOP), which is outside our current scope.

Instruction - Python Basics

Operators & Expressions

What are they?

In Python, **operators** are special symbols that perform operations on variables and values. **Expressions** are combinations that produce a result.

Think of operators as tools to manipulate data, just like a calculator!

Arithmetic Operators

- `+` , `-` , `*` , `/` (Standard Math)
- `//` Floor Division (removes decimal)
- `%` Modulus (remainder)
- `**` Exponentiation

Try running these in a Jupyter Notebook!

Example Code

```
x = 10
y = 3

# Prints results
print("Addition:", x + y) # 13
print("Division:", x / y) # 3.333
print("Floor Div:", x // y) # 3
print("Modulus:", x % y) # 1
print("Exponent:", x ** y) # 1000
```

Instruction - Python Basics

Comparison Operators

Comparison operators compare two values and return **True** or **False**. They are the foundation of decision-making in code.

Quick Reference Table

Operator	Meaning	Example
==	Equal to	`5 == 5`
!=	Not equal to	`5 != 3`
>	Greater than	`10 > 3`
<	Less than	`2 < 8`
>=	Greater or equal	`6 >= 6`
<=	Less or equal	`4 <= 5`

Comparison results are always Boolean (True/False).

Example Code

```
a = 5
b = 10

print(a == b) # False
print(a != b) # True
print(a > b) # False
print(a < b) # True
```

Importance:
Essential for making decisions in your code (e.g., if statements).

Instruction - Python Basics

Logical Operators

Logical operators combine multiple conditions to return **True** or **False**. They allow for complex decision-making logic.

Key Operators

`and`
Returns True if both statements are true.

`or`
Returns True if one of the statements is true.

`not`
Reverse the result, returns False if the result is true.

Why It Matters:
Helpful when coding decision-making structures like if statements.

Example Code

```
x = 5
y = 10

print(x > 3 and y > 5) # True

print(x > 3 or y < 5) # True

print(not(x > 3)) # False
```

Instruction - Python Basics

Assignment Operators

Assignment operators are used to **assign values to variables** and perform shorthand arithmetic updates in a single step.

Quick Reference

`=` Assign: `x = 5`

`+=` Add: `x = x + 3`

`-=` Sub: `x = x - 2`

`*=` Mul: `x = x * 4`

`/=` Div: `x = x / 2`

Pro Tip:
They help us update variables efficiently without re-typing the variable name.

Example Code

```
x = 10
x += 5 # x = x + 5
print(x) # Output: 15

y = 20
y /= 4 # y = y / 4
print(y) # Output: 5.0
```

Instruction - Python Basics

Expressions in Python

An **expression** is a combination of variables, values, and operators that produces a result. Every time Python evaluates an expression, it **returns a value**.

Examples of Expressions

```
result = (10 + 2) * 3  # 36

is_greater = 15 > 10  # True

status = (5 > 3) and (8 < 10)  # True
```

Test It Out: Try creating your own expressions in a Jupyter Notebook!

Summary

Operators help us perform calculations and comparisons.

Expressions combine values and operators to produce results.

Understanding these concepts will help you write powerful programs!

Next Steps

Now move on to the next activity to practice these operators and expressions.

There will be a small quiz on this after the activity.

Activity - Math and Logic in Python

Activity Overview

Objectives

By the end of this activity, you will:

- Use arithmetic operators to perform calculations in Python.
- Apply comparison and logical operators to make decisions.
- Write and evaluate expressions using Python code.

Instructions

Complete a series of coding tasks to practice using **math and logic in Python**.

Steps:

1. Open your Jupyter Notebook.
2. Follow the task prompts carefully.
3. Write and test your code for each section.

Activity - Math and Logic in Python

Operators Practice

Part 1: Arithmetic Operators

Let's start by performing some basic math operations!

Task:

- Create variables a, b
- Use +, -, *, /, //, %, **
- Print the results

Example Code

```
a = 10; b = 3
print("Addition:", a + b)
print("Division:", a / b)
print("Modulus:", a % b)
```

Try it yourself! Modify the values and observe changes.

Part 2: Comparison Operators

Comparison operators return True or False results.

Task:

- Choose variables x, y
- Use ==, !=, >, <, >=, <=
- Print the comparison results

Example Code

```
x = 7; y = 10
print(x == y) # False
print(x != y) # True
print(x < y) # True
```

Reminder: Use these to help in your future activities.

Activity - Math and Logic in Python

Logical Operators & Expressions

Part 3: Logical Operators

Combine multiple conditions using **and, or, not**.

Task:

- Pick two comparison statements
- Print results for different operators

Example Code

```
x = 5; y = 10
print((x > 3) and (y > 5)) # True
print((x > 10) or (y < 15)) # True
print(not(x > 3)) # False
```

Experiment with different values!

Part 4: Writing Expressions

Combine everything we've learned into a program.

Task:

- Calculate student "Pass" or "Fail"
- Threshold: score >= 70
- Bonus: Add attendance >= 80%

Example Code

```
score = 85; attendance = 90
if (score >= 70) and (attendance >= 80):
        print("Pass")
Else:
        print("Fail")
```

Test different cases!

Summary

Arithmetic operators help perform math calculations.

Comparison operators compare values and return True or False.

Logical operators combine conditions to make decisions.

Expressions allow us to compute values and determine outcomes in programs.

Submission Requirement

Submit your Jupyter Notebook for credit.

Activity - Math and Logic in Python

Create a Simple Calculator

Programming Task

Write a Python program that asks for two numbers and an operator (+, -, *, /).

Requirements:

- Perform calculations based on the operator.
- Display the final result.
- Use input() and if statements.

Expected Output

How your program should look:

```
Enter first number: 8
Enter operator: *
Enter second number: 5

Result: 40
```

Tip: Ensure your code handles all four operators.

Submission: Submit your Jupyter Notebook to this assignment for full credit.

Activity - Math and Logic in Python

Activity - Tip Calculator

Programming Task

In this activity, you will be tasked with creating a tip calculator program in Python.

Requirements:

- You should be able to hit the run-all button.
- It should prompt someone for their bill.
- It should prompt for how much they want to tip.
- It should display the total amount to leave.

Expected Interaction

Example of how your program should run:

```
Enter bill amount: 50.00
Enter tip percentage: 20
Total amount to leave: $60.00
```

Tip: Convert input to float to handle decimals.

Submission: Submit your Jupyter Notebook to this assignment for full credit.

Why This Matters

You just built programs that:

- **Take input:** Interaction with the user.
- **Apply logic:** Processing data based on rules.
- **Produce output:** Delivering meaningful results.

This is the foundation of how data-driven systems work.